



Daffodil
International
University

Daffodil International University

Department of Computing and Information System

Faculty of Science & Information Technology

Semester Project, Spring-2024

Course Title: Artificial Intelligence lab

SUBMITTED TO:

Dr.Md Mizanur Rahman

Lecturer,

Department of Computing Information and System (CIS).

Daffodil International University (DIU).

SUBMITTED BY:

Ram Prosad kumar mohanto

ID:212-16-571

Department of Computing Information and System (CIS).

Daffodil International University (DIU).

1. Software Development:

Software Development for Enhanced Bus Passenger Tracking and Distance Calculation System

Implement OpenCV for Passenger Counting

1. Video Analysis Setup:
 - Onboard Camera Integration: Install and configure cameras in strategic locations on the bus to capture video footage of passengers boarding and alighting.
 - Video Stream Processing: Develop software to capture and process real-time video streams from the onboard cameras.
2. Passenger Detection and Counting:
 - Model Selection: Use pre-trained machine learning models such as YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), or Faster R-CNN for object detection to identify passengers.
 - Frame Analysis: Implement OpenCV to analyze each frame of the video stream. Use object detection models to detect and count passengers entering and exiting the bus.
 - Tracking Algorithms: Employ tracking algorithms like SORT (Simple Online and Realtime Tracking) or Deep SORT to maintain a count of passengers over consecutive frames, ensuring accurate counting even when multiple passengers move simultaneously.
3. Data Handling:
 - Event Logging: Log events of passengers boarding and alighting with corresponding timestamps.

Integrate GPS Data Processing

1. GPS Module Integration:
 - Hardware Configuration: Set up a GPS module on the bus to continuously capture location data.
 - Data Retrieval: Develop software to retrieve and store real-time GPS data, including latitude, longitude, and timestamp.
2. Route Tracking:
 - Data Synchronization: Synchronize GPS data with the video analysis timestamps to correlate passenger boarding and alighting events with specific locations.
 - Path Recording: Record the bus's path throughout the journey, capturing the GPS coordinates at regular intervals.

Calculate Individual Travel Distances

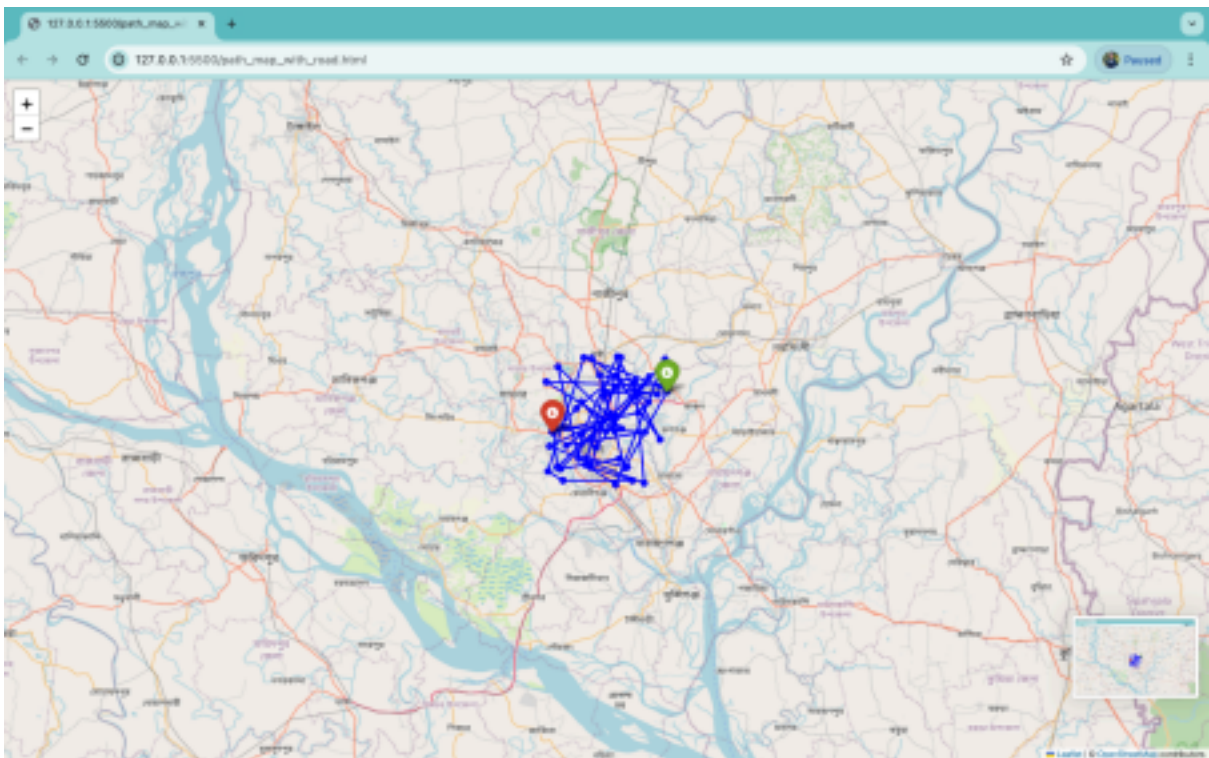
1. Boarding and Alighting Event Correlation:
 - Event Matching: Match boarding and alighting events recorded by the video analysis system with the corresponding GPS data.
 - Passenger Identification: Assign unique identifiers to passengers based on detected events to track their entire journey from boarding to alighting.
2. Distance Calculation Algorithm:

- Path Segmentation: Segment the bus's path into sections between consecutive GPS points.
 - Distance Measurement: Calculate the distance traveled between GPS points using the Haversine formula or other suitable methods to account for Earth's curvature.
 - Individual Distance Calculation: Sum the distances of path segments corresponding to each passenger's journey to determine the total distance traveled by each passenger.
3. Data Management:
- Database Integration: Store all relevant data, including passenger counts, GPS coordinates, and calculated distances, in a database for further analysis and reporting.
 - Real-Time Updates: Ensure real-time updates and accessibility of data for monitoring and operational purposes.

System Integration and Testing

1. Integration:
- Module Coordination: Ensure seamless integration of the video processing, GPS data processing, and distance calculation modules.
 - Communication Protocols: Establish communication protocols between different components of the system to ensure data consistency and accuracy.
2. Testing and Validation:
- Simulated Environment Testing: Conduct extensive testing in a simulated environment to validate the accuracy and reliability of the passenger counting and distance calculation algorithms.
 - Field Testing: Perform field tests on actual bus routes to verify the system's performance in real-world conditions.
3. Optimization:
- Performance Tuning: Optimize the algorithms and software components for real-time performance and minimal computational overhead.
 - Accuracy Enhancement: Continuously refine the models and algorithms based on field test results to enhance counting accuracy and distance calculation precision.

By following these steps, the development of the Enhanced Bus Passenger Tracking and Distance Calculation System can be effectively achieved, leading to improved operational efficiency and enhanced service quality for public transportation systems.



```
import face_recognition
```

```
import cv2
import numpy as np
```

```
import uuid
```

```
#import os
```

```

from mtcnn import MTCNN

# from detect_face import detect_face, extract_face

import folium

import random

import numpy as np

from scipy.spatial import distance

# Function to generate random GPS coordinates within a specified area

def generate_random_coordinates(center, radius, num_points):

    random_coordinates = []

    for _ in range(num_points):

        lat = center[0] + random.uniform(-radius, radius)

        lon = center[1] + random.uniform(-radius, radius)

        random_coordinates.append([lat, lon])

    return random_coordinates

# Calculate distance between two coordinates in kilometers

def calculate_distance(coord1, coord2):

    R = 6371.0 # Earth radius in kilometers

    lat1, lon1 = np.radians(coord1)

    lat2, lon2 = np.radians(coord2)

    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = np.sin(dlat / 2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2

```

```

c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))

distance = R * c

return distance


# Initialize map centered at a specific location

center_location = [23.8103, 90.4125] # Dhaka, Bangladesh

mymap = folium.Map(location=center_location, zoom_start=10)


# Generate synthetic GPS coordinates

num_points = 50 # Number of synthetic GPS points

radius = 0.1 # Radius (in degrees) around the center location

synthetic_coordinates = generate_random_coordinates(center_location, radius,
num_points)


# Add synthetic GPS markers to the map

for i, coord in enumerate(synthetic_coordinates):

    folium.CircleMarker(location=coord, radius=3, color='blue',
fill=True, fill_color='blue').add_to(mymap)


# Define start and end locations

start_location = synthetic_coordinates[0]

end_location = synthetic_coordinates[-1]


# Add start and end markers to the map
folium.Marker(location=start_location, popup='Start Location',
icon=folium.Icon(color='green')).add_to(mymap)

folium.Marker(location=end_location, popup='End Location',
icon=folium.Icon(color='red')).add_to(mymap)

```

```

# Draw path between all points from start to end and calculate total distance

path_points = [start_location] + synthetic_coordinates + [end_location]

folium.PolyLine(locations=path_points, color='blue', weight=2.5,
opacity=1).add_to(mymap)


# Save the map to an HTML file

mymap.save('path_map_with_road.html')


# Print total distance in kilometers

total_distance_km = sum(calculate_distance(path_points[i], path_points[i + 1]) for
i in range(len(path_points) - 1))

#print("Total distance (kilometers):", total_distance_km)
def generate_id():

    return str(uuid.uuid4())


def calculate_sharpness(image):

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    laplacian = cv2.Laplacian(gray, cv2.CV_64F).var()

    return laplacian


# This is a demo of running face recognition on live video from your webcam. It's a
little more complicated than the

# other example, but it includes some basic performance tweaks to make things run a
lot faster:

# Only detect faces in every other frame of video.

```

```
# PLEASE NOTE: This example requires OpenCV (the `cv2` library) to be installed
only to read from your webcam.

# OpenCV is *not* required to use the face_recognition library. It's only required
if you want to run this

# specific demo. If you have trouble installing it, try any of the other demos that
don't require it instead.


# Get a reference to webcam #0 (the default one)

video_capture = cv2.VideoCapture(0)

by_pass_cap = cv2.VideoCapture(0)


file_obama_path = "obama.jpeg"
# file_obama_path = r'..image\reference.jpg'


obama_image = face_recognition.load_image_file(file_obama_path) #

obama_image = face_recognition.load_image_file("obama.jpg")

obama_face_encoding = face_recognition.face_encodings(obama_image)[0]


# Create arrays of known face encodings and their
names known_face_encodings = [

    obama_face_encoding

]

known_face_names = [

    'Mr. Reference'

]
```



```

# Initialize some variables

face_locations = []

face_encodings = []

face_names = []

process_this_frame = True


person_count = 0


# Define the coordinates of the region of interest (ROI) # Format:
(x1, y1) = Top-left corner, (x2, y2) = Bottom-right corner x1, y1 =
10, 110 # Top-left corner

x2, y2 = 620, 400 # Bottom-right corner
# initialize the MTCNN detector

detector = MTCNN()


font = cv2.FONT_HERSHEY_DUPLEX


while True:

    # Grab a single frame of video

    ret, frame = video_capture.read()


    roi = frame[y1:y2, x1:x2]

    # Draw the roi around the fame

```

```

cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

text = f"count {person_count} distance {total_distance_km:.2f} fare
{(total_distance_km * 5 * person_count):.2f}"

cv2.putText(frame, text, (5, 100), font, 1.0, (0, 0, 255), 1)

# Only process every other frame of video to save time

if process_this_frame : # and calculate_sharpness(roi) > 30:

    # Convert the image from BGR color (which OpenCV uses) to RGB color (which
    face_recognition uses)

    # rgb_small_frame = np.ascontiguousarray(small_frame[:, :, ::-1])
    rgb_roi = np.ascontiguousarray(roi[:, :, ::-1])

    # Find all the faces and face encodings in the current frame of video

    # detect faces using MTCNN

    faces = detector.detect_faces(roi)

    face_locations = []

    for face in faces :

        x, y, w, h = face['box']

```

```

        face_locations.append([y, x+w, y+h, x])

    # face_locations = face_recognition.face_locations(rgb_roi)

    face_encodings = face_recognition.face_encodings(rgb_roi,
face_locations,num_jitters=5,model='large')

    face_names = []

    # for face_encoding in face_encodings:

    for face_location, face_encoding in zip(face_locations, face_encodings):

        top, right, bottom, left = face_location

        # top = y1, right = x2, bottom = y2, left = x1
        height, width, channels = roi[top:bottom, left:right].shape

        if height <= 100 or width <= 100:

            continue

        # See if the face is a match for the known face(s)

        matches = face_recognition.compare_faces(known_face_encodings,
face_encoding, tolerance=0.65)

        print(matches)

        # # If a match was found in known_face_encodings, just use the first
one.

        # if True in matches:

        # first_match_index = matches.index(True)

        # name = known_face_names[first_match_index]

```

```

        # Or instead, use the known face with the smallest distance to the new
face

        face_distances = face_recognition.face_distance(known_face_encodings,
face_encoding)

        print(face_distances)

        # exit(0)

        best_match_index = np.argmin(face_distances)

        if matches[best_match_index]:

            name = known_face_names[best_match_index]

            print(name)

            # # exit(0)

            face_names.append(name)
        else :

            person_count += 1

            unkown_per_name = str(uuid.uuid4())

            filename = r'.\image' + '\\ ' + unkown_per_name + '.jpg'

            # cv2.imwrite(filename, rgb_small_frame)

            top, right, bottom, left = face_location

            # top = y1, right = x2, bottom = y2, left = x1

            cv2.imwrite(filename, roi[top:bottom, left:right])

            known_face_encodings.append(face_encoding)

            known_face_names.append(unkown_per_name)

```

```

        face_names.append(unkown_per_name)

    process_this_frame = not process_this_frame


    # Display the results

    for (top, right, bottom, left), name in zip(face_locations, face_names):

        # Draw a box around the face

        cv2.rectangle(frame, (left, top+35), (right, bottom+100), (0, 0, 255), 2)


        # Draw a label with a name below the face


        # cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0, 255),
cv2.FILLED)

        name_last_part = name.split('-')[-1]

        cv2.putText(frame, "name :" + name_last_part, (left + 6, bottom+25), font,
1.0, (255, 255, 255), 1)


        print(f"faces in the capture {len(face_names)} person count
{person_count}")


    # Display the resulting image

    cv2.imshow('Video', frame)

```

```
# Hit 'q' on the keyboard to quit!

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

# Release handle to the webcam

video_capture.release()

cv2.destroyAllWindows()
```

2.Machine Learning Model Training:

Collect labeled data: Acquire a comprehensive dataset of video footage capturing passengers boarding and alighting from buses, each annotated with the exact number of passengers. This dataset should cover a wide range of conditions, including different times of day, various weather conditions, and varying passenger loads.

- **Train machine learning model:** Develop a machine learning model using the labeled dataset to accurately count passengers. This involves selecting an appropriate algorithm, training the model with the annotated data, and performing extensive testing and validation. Fine-tune the model to handle challenges such as changes in lighting conditions, passenger movement patterns, and varying passenger densities, ensuring robust and reliable performance across different scenarios.

Documentation:

System Architecture

1. Overview

- Describe the purpose and objectives of the Enhanced Bus Passenger Tracking and Distance Calculation System.
- Highlight key components: onboard cameras, GPS module, software modules (OpenCV for passenger counting, GPS data processing).

2. Hardware Requirements

- Specify the hardware components needed:
 - Onboard cameras: Type, placement, and connectivity requirements.
 - GPS module: Type (e.g., GPS receiver), mounting instructions, and connectivity to the system.

3. Software Components

- Detail the software modules and their functionalities:
 - OpenCV for real-time passenger counting from video footage.
 - GPS data processing software for location tracking and distance calculation.
 - Integration methods and protocols between software components.

4. System Integration

- Explain how hardware and software components interact:
 - Data flow from video capture to passenger counting and GPS-based distance calculation.
 - Synchronization mechanisms to ensure accurate data correlation between video analysis and GPS tracking.

Setup Instructions

1. Installation

- Step-by-step guide for installing onboard cameras:
 - Mounting instructions and placement recommendations.
 - Configuration of camera settings for optimal video capture.

2. GPS Module Setup

- Instructions for installing and configuring the GPS module:
 - Mounting guidelines and antenna placement.
 - Configuration of GPS module settings and connectivity to the system.

3. Software Installation

- Installation procedures for:
 - OpenCV libraries and dependencies for video analysis.
 - GPS data processing software and drivers for data retrieval.

4. System Configuration

- Configuration steps to:
 - Configure video analysis parameters (e.g., frame rate, resolution).
 - Set up GPS data synchronization intervals and data processing settings.

Operational Procedures

1. Startup and Shutdown

- Guidelines for starting up and shutting down the system:
 - Sequence of operations to power on/off cameras, GPS module, and software components.
 - Checks and tests to perform during startup to ensure system readiness.
- 2. Routine Maintenance**
 - Maintenance procedures to keep the system operational:
 - Cleaning and maintenance of onboard cameras.
 - Inspection and calibration of GPS module and antenna.
- 3. Data Management**
 - Procedures for managing data collected by the system:
 - Storage location and organization of video footage and GPS data.
 - Backup and archival procedures for data security and integrity.

Training for Operators

- 1. Training Sessions Outline**
 - Overview of the system's purpose and benefits for public transportation services.
 - Introduction to hardware components (cameras, GPS module) and their functions.
 - Demonstration of software functionalities (OpenCV for passenger counting, GPS data processing).
- 2. Operational Training**
 - Hands-on training for bus operators:
 - Using the system interface to monitor passenger counts and journey progress.
 - Troubleshooting common issues related to video capture, GPS data synchronization, and software operation.
- 3. Adoption and Feedback**
 - Encouragement for feedback and questions from operators during training sessions.
 - Provision of contact information for technical support and assistance post-training.

By following these comprehensive documentation and training guidelines, we aim to ensure the successful implementation and adoption of the Enhanced Bus Passenger Tracking and Distance Calculation System. This approach will facilitate improved public transportation services, optimized route planning, and enhanced operational efficiency for a better commuting experience.